

[Algorithm]

최근접 점의 쌍 찾기 알고리즘

인하대학교 소프트웨어융합공학과

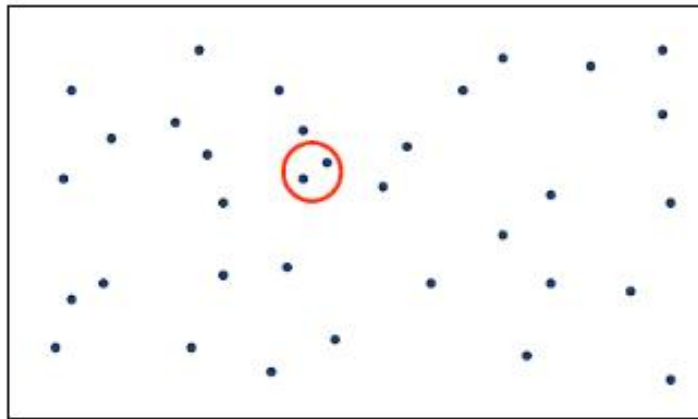
김종훈 교수



인하대학교
INHA UNIVERSITY

최근접 점의 쌍 찾기

- 최근접 점의 쌍 (Closest Pair) 문제는 2차원 평면상의 n 개의 점이 입력으로 주어질 때, 거리가 가장 가까운 한 쌍의 점을 찾는 문제



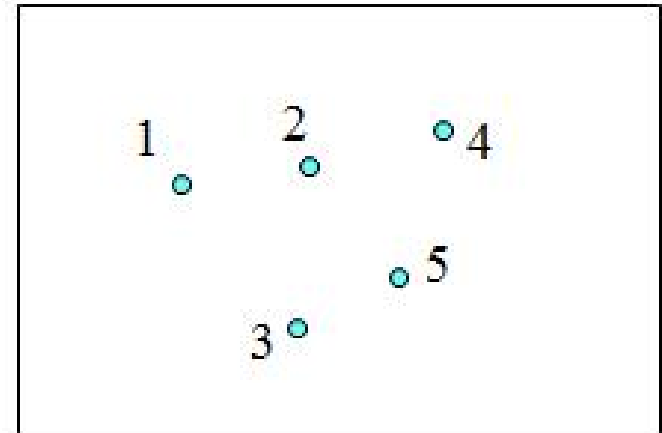
최근접 점의 쌍 찾기

□ 간단한 방법

- 모든 점에 대하여 각각의 두 점 사이의 거리를 계산하여 가장 가까운 점의 쌍을 찾는다.
- 예를 들어, 5개의 점이 아래의 [그림]처럼 주어지면, 1-2, 1-3, 1-4, 1-5, 2-3, 2-4, 2-5, 3-4, 3-5, 4-5 사이의 거리를 각각 계산하여 그 중에 최소 거리를 가진 쌍이 최근접 점의 쌍이 되는 것이다.

- 비교해야 할 쌍은 몇 개인가?

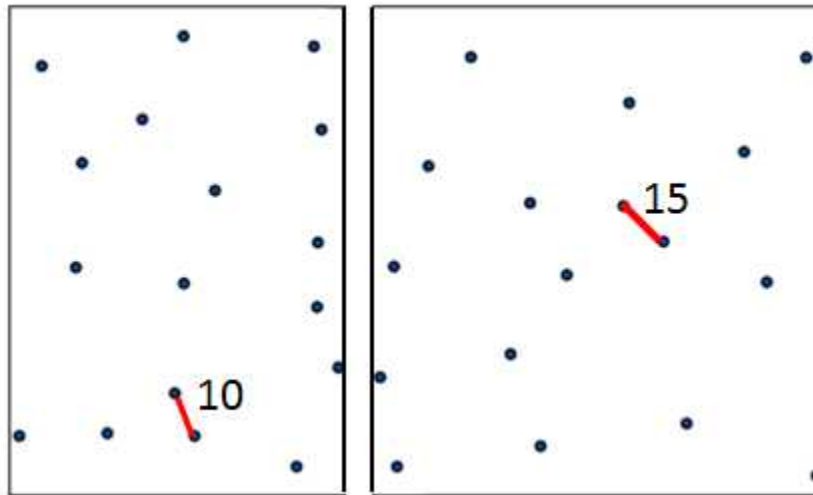
- ${}_nC_2 = n(n-1)/2$
- $n=5$ 이면, $5(5-1)/2 = 10$
- $n(n-1)/2 = O(n^2)$
- 한 쌍의 거리 계산은 $O(1)$
- 시간복잡도는 $O(n^2) \times O(1) = O(n^2)$



최근접 점의 쌍 찾기

□ $O(n^2)$ 보다 효율적인 분할 정복 이용

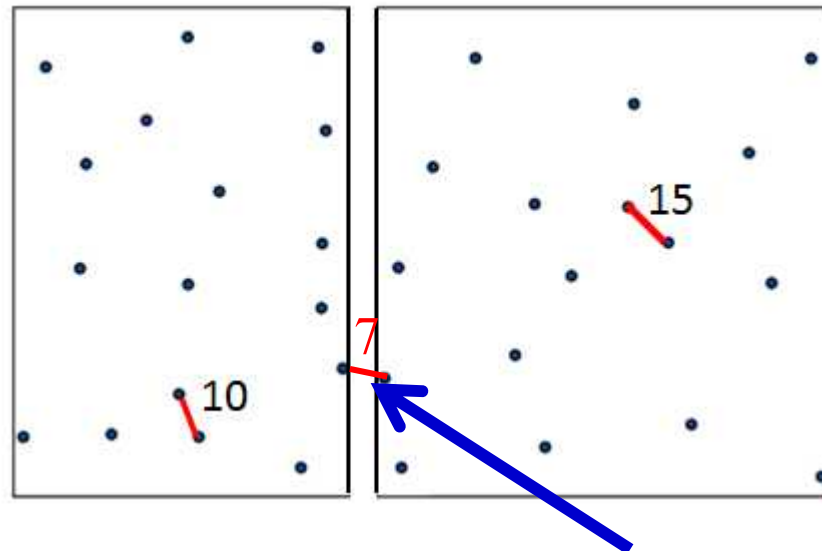
- n 개의 점을 1/2로 분할하여 각각의 부분 문제에서 최근접 점의 쌍을 찾고, 2개의 부분해 중에서 짧은 거리를 가진 점의 쌍을 일단 찾는다.



- 그리고 2개의 부분해를 취합할 때에는 반드시 다음과 같은 경우를 고려해야 한다.

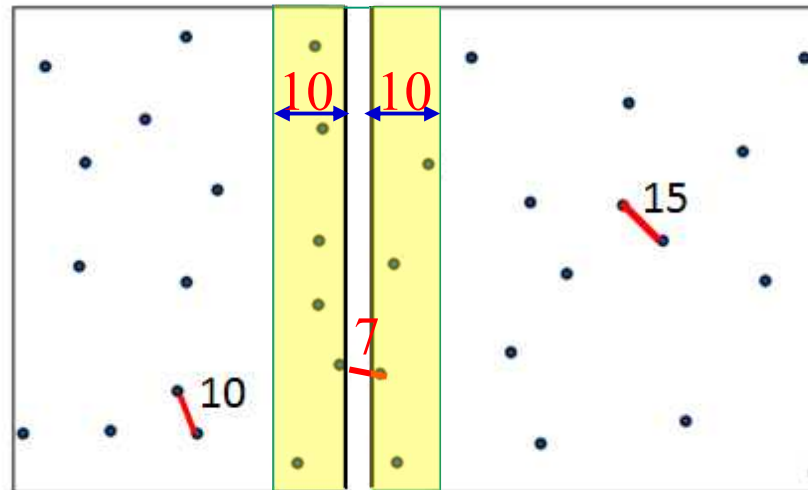
최근접 점의 쌍 찾기

- 왼쪽 부분 문제의 최근접 쌍의 거리가 10이고, 오른쪽 부분 문제의 최근접 쌍의 거리가 15이다.
- 왼쪽 부분 문제의 가장 오른쪽 점과 오른쪽 부분 문제의 가장 왼쪽 점 사이의 거리가 7이다.
- 따라서 2개의 부분 문제의 해를 취합할 때 단순히 10과 15 중에서 짧은 거리인 10을 해라고 할 수 없는 것이다.



최근접 점의 쌍 찾기

- 그러므로 아래의 그림에서와 같이 각각 거리가 10이내의 중간 영역 안에 있는 점들 중에 최근접 점의 쌍이 있는지도 확인해보아야 한다.



최근접 점의 쌍 찾기

- 배열에 점의 좌표가 저장되어 있을 때, 중간 영역에 있는 점들을 찾는 방법
 - 단, $d = \min\{\text{왼쪽 부분의 최근접 점의 쌍 사이의 거리}, \text{오른쪽 부분의 최근접 점의 쌍 사이의 거리}\}$ 이다.
 - 아래의 배열에는 점들이 x-좌표의 오름차순으로 정렬되어 있고, 각 점의 y-좌표는 생략되었다.

왼쪽 부분 문제의 가장오른쪽 점						오른쪽 부분 문제의 가장왼쪽 점			
(1,-)	(13,-)	(17,-)	(25,-)	(26,-)	(28,-)	(30,-)	(37,-)	(45,-)	(56,-)

최근접 점의 쌍 찾기

- 중간 영역에 속한 점들은 왼쪽 부분 문제의 가장 오른쪽 점 (왼쪽 중간점)의 x-좌표에서 d 를 뺀 값과 오른쪽 부분 문제의 가장 왼쪽 점 (오른쪽 중간점)의 x-좌표에 d 를 더한 값 사이의 x-좌표 값을 가진 점들이다.
- $d=10$ 이라면, 점 $(17,-)$, $(25,-)$, $(26,-)$, $(28,-)$, $(30,-)$, $(37,-)$ 이 중간 영역에 속한다.

$d = 10$

$(1,-)$	$(13,-)$	$(17,-)$	$(25,-)$	$(26,-)$	$(28,-)$	$(30,-)$	$(37,-)$	$(45,-)$	$(56,-)$

$$26-d = 16$$

$$28+d = 38$$

최근접 점의 쌍 분할 정복 알고리즘

ClosestPair(S)

입력: x-좌표의 오름차순으로 정렬된 배열 S에 있는 i개의 점
(단, 각 점은 (x,y)로 표현된다.)

출력: S에 있는 점들 중 최근접 점의 쌍의 거리

1. if ($i \leq 3$) return (2 또는 3개의 점들 사이의 최근접 쌍)
2. 정렬된 S를 같은 크기의 S_L 과 S_R 로 분할한다. |S|가 홀수이면, $|S_L| = |S_R| + 1$ 이 되도록 분할한다.
3. $CP_L = \text{ClosestPair}(S_L)$ // CP_L 은 S_L 에서의 최근접 점의 쌍
4. $CP_R = \text{ClosestPair}(S_R)$ // CP_R 은 S_R 에서의 최근접 점의 쌍
5. $d = \min\{\text{dist}(CP_L), \text{dist}(CP_R)\}$ 일 때, 중간 영역에 속하는 점들 중에서 최근접 점의 쌍을 찾아서 이를 CP_C 라고 하자.
단, $\text{dist}()$ 는 두 점 사이의 거리이다.
6. return (CP_L, CP_C, CP_R 중에서 거리가 가장 짧은 쌍)

최근접 점의 쌍 분할 정복 알고리즘

□ Line 1

- S 에 있는 점의 수가 3개 이하이면 더 이상 분할하지 않는다.
- S 에 2개의 점이 있으면 S 를 그대로 리턴하고, 3개의 점이 있으면 3개의 쌍에 대하여 최근접 점의 쌍을 리턴한다.

□ Line 2

- x -좌표로 정렬된 S 를 왼쪽과 오른쪽에 같은 개수의 점을 가지는 S_L 과 S_R 로 분할한다.
- 만일 S 의 점의 수가 홀수이면 S_L 쪽에 1개 많게 분할한다.

□ Line 3~4

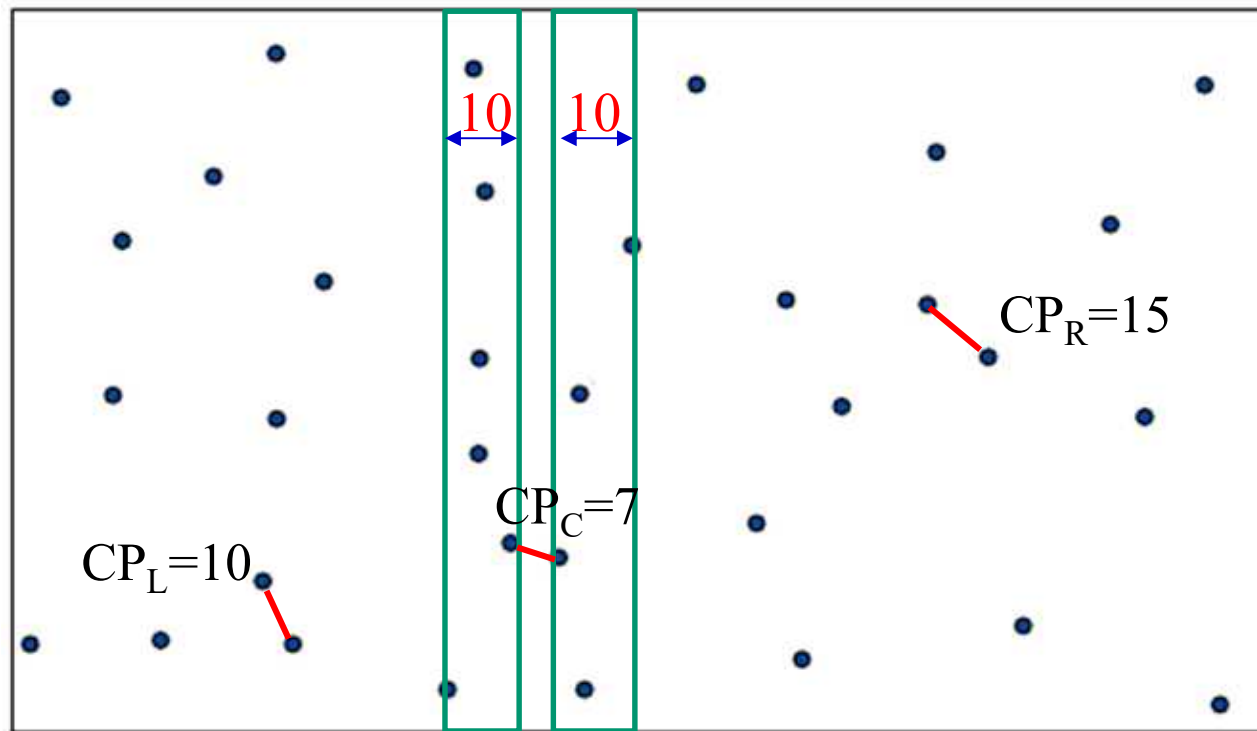
- 분할된 S_L 과 S_R 에 대해서 재귀적으로 최근접 점의 쌍을 찾아서 각각을 CP_L 과 CP_R 이라고 놓는다.

최근접 점의 쌍 분할 정복 알고리즘

□ Line 5

- d 를 이용하여 중간 영역에 속하는 점들을 찾고, 이 점들 중에서 최근접 점의 쌍을 찾아서 이를 CP_C 라고 놓는다.

$$d = \min \{CP_L, CP_R\} = \min \{10, 15\} = 10$$



최근접 점의 쌍 분할 정복 알고리즘

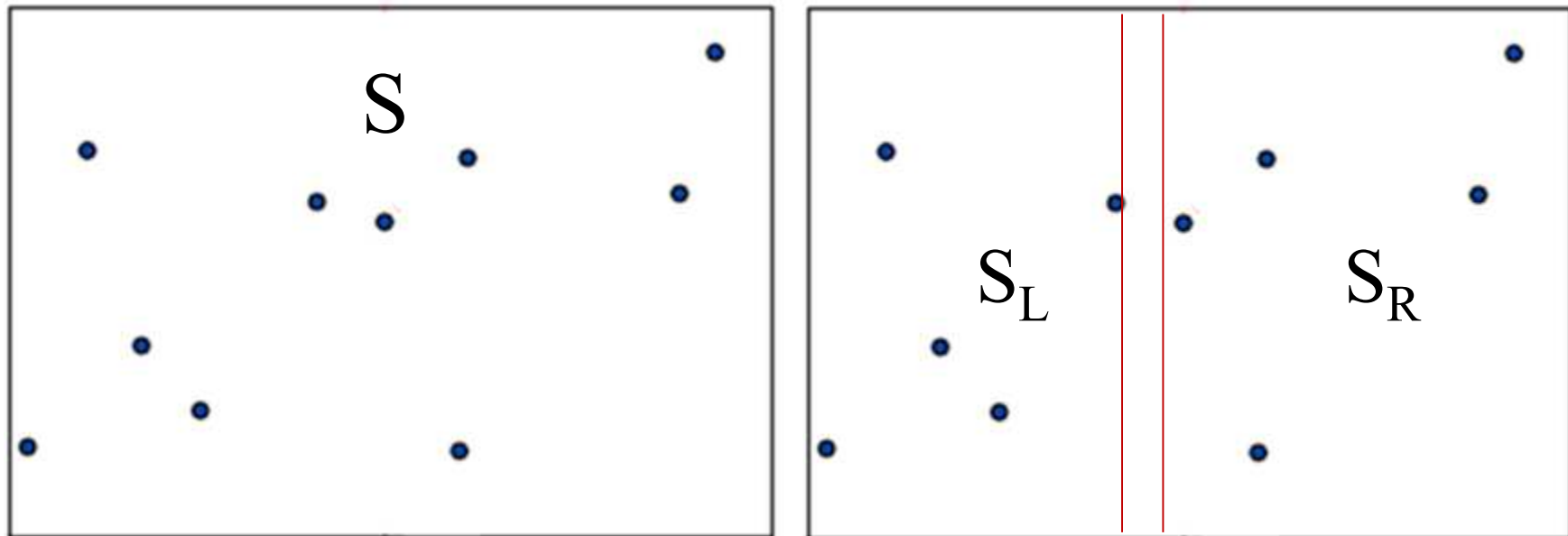
□ Line 6

- line 3~4에서 각각 찾은 최근접 점의 쌍 CP_L 과 CP_R 과 line 5에서 찾은 CP_C 중에서 가장 짧은 거리를 가진 쌍을 해로서 리턴한다.

ClosestPair의 수행 과정

□ ClosestPair(S)로 호출 [1]

- 단, S는 아래의 그림에 주어진 점들
- Line 1: S의 점의 수 > 3이므로 다음 line을 수행
- Line 2: S를 S_L 과 S_R 로 분할

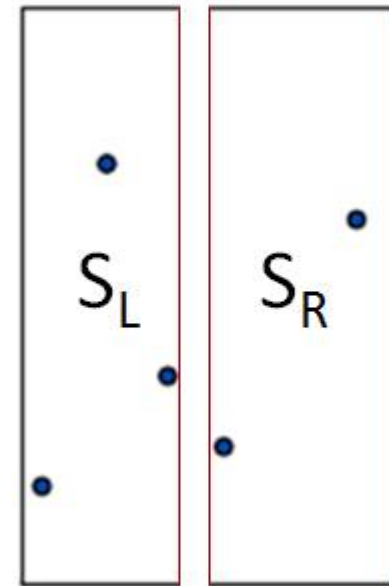


ClosestPair의 수행 과정

- Line 3: ClosestPair(S_L) 호출
 - ClosestPair(S_L)을 수행한 후 리턴된 점의 쌍을 CP_L 이라고 놓은 후에 line 4~6을 차례로 수행

□ ClosestPair(S_L) 호출 [2]

- Line 1의 if-조건이 '거짓'이므로 line 2에서 다시 분할
- Line 3: ClosestPair(S_L) 호출
(여기서의 S_L 은 처음 S_L 의 왼쪽 반)
- ClosestPair(S_L)을 수행한 후, 리턴된 점의 쌍을 CP_L 이라고 놓은 후에 line 4~6을 차례로 수행



ClosestPair의 수행 과정

□ ClosestPair(S_L) 호출

- Line 1의 if-조건이 '참'; S_L 의 3개의 점들에 대해서 최근접 점의 쌍을 찾는다. 아래 그림과 같이 3개의 쌍에 대해 거리를 각각 계산하여 최근접 쌍을 해로서 리턴
- 최근접 점의 쌍의 거리를 20이라고 가정



ClosestPair의 수행 과정

□ ClosestPair(S_R) 호출

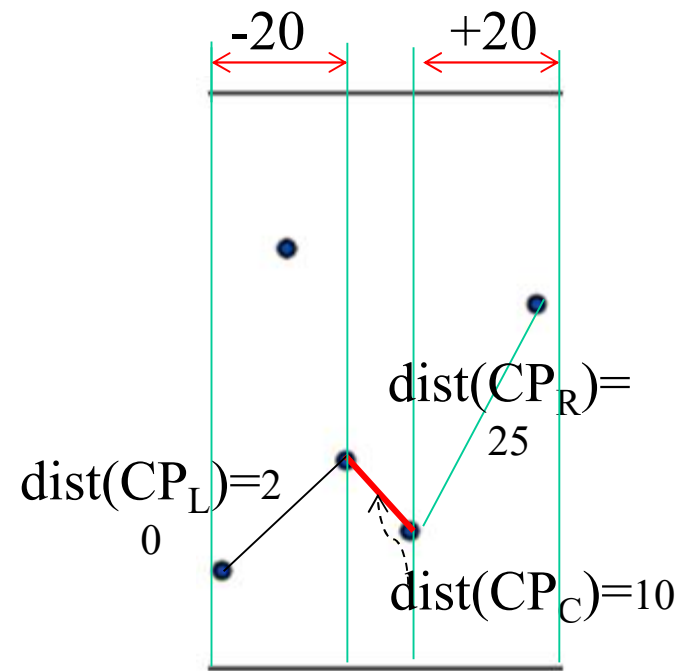
- Line 1에서 점의 수가 2이므로, 이 두 점을 최근접 점의 쌍으로 리턴
- 최근접 점의 쌍의 거리를 25라고 가정



ClosestPair의 수행 과정

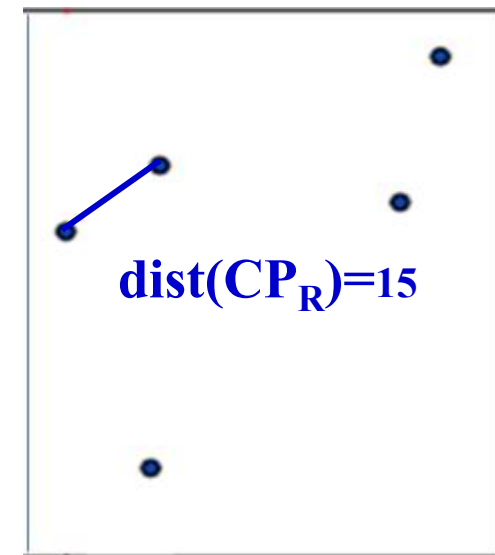
- [2]의 ClosestPair(S_L) 호출 당시 line 3~4가 수행되었고, 이제 line 5가 수행된다.
- Line 3~4에서 찾은 최근접 점의 쌍 사이의 거리인 $\text{dist}(CP_L)=20$ 과 $\text{dist}(CP_R)=25$ 중에 작은 값을 $d=20$ 라고 놓는다.

- 그리고 왼쪽 중간점의 x-좌표에서 20을 뺀 값과 오른쪽 중간점의 x-좌표에 20을 더한 값 사이의 x-좌표 값을 가진 점들 중에서 CP_C 를 찾는다.
- 옆의 그림과 같이 거리가 10인 CP_C 를 찾는다.



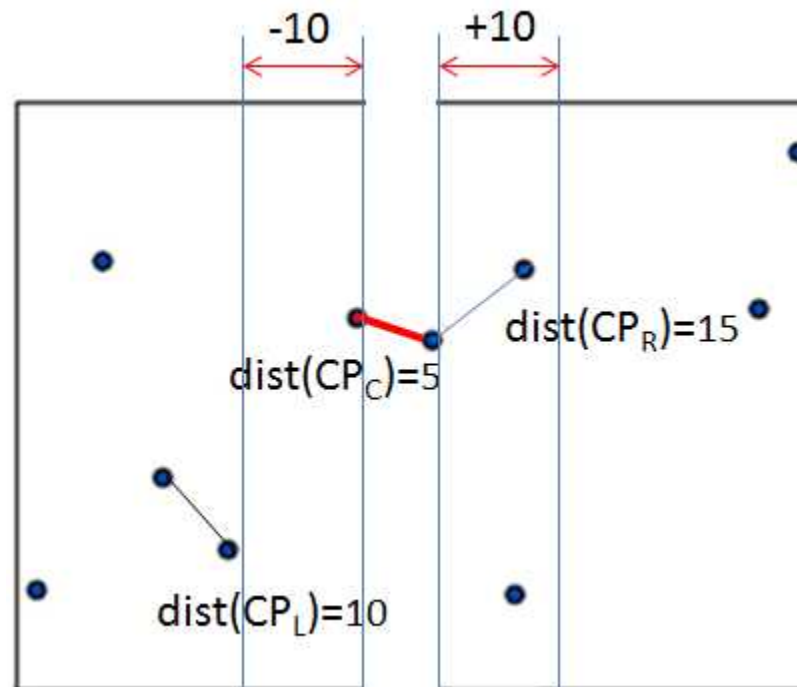
ClosestPair의 수행 과정

- Line 6: $\text{dist}(CP_L)=20$, $\text{dist}(CP_C)=10$, $\text{dist}(CP_R)=25$
중에서 가장 거리가 짧은 쌍인 $CP_C=10$ 을 리턴
- [1]의 $\text{ClosestPair}(S)$ 호출 당시 line 3이 수행되었고,
이제 line 4에서는 $\text{ClosestPair}(S_R)$ 을 호출한다.
 - 여기서 S_R 은 초기 입력의 오른쪽 반인 영역이다.
 $\text{ClosestPair}(S_R)$ 호출 결과로 옆 그림의 최근접 점의 쌍을
리턴하고 이를 CP_R 로 놓는다.
 - 이때 $\text{dist}(CP_R)=15$ 라고 하자.



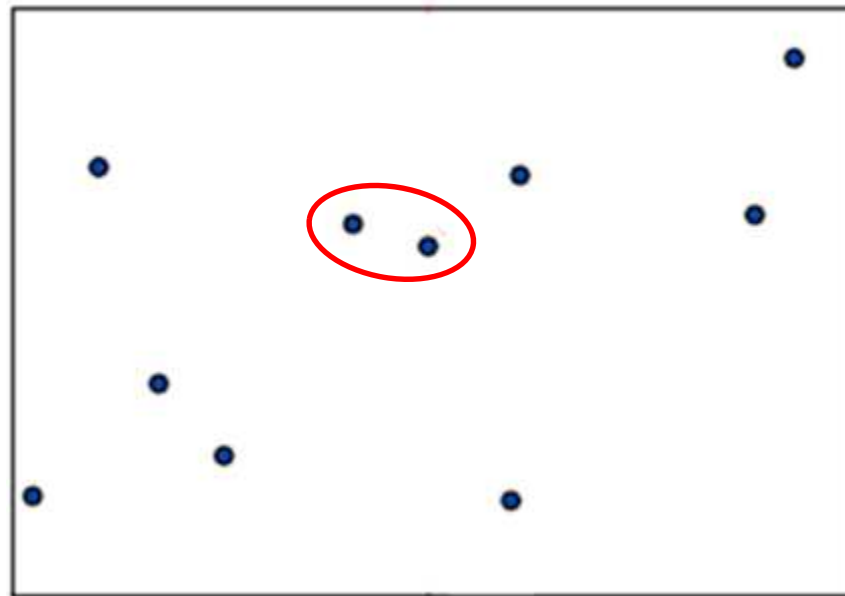
ClosestPair의 수행 과정

- Line 5: line 3~4에서 찾은 최근접 점의 쌍 사이의 거리인 $\text{dist}(\text{CP}_L)=10$ 과 $\text{dist}(\text{CP}_R)=15$ 중에 작은 값을 $d=10$ 이라고 놓는다.
- 그리고 중간 영역에 있는 점들 중에서 CP_C 를 찾는다. 여기서는 아래의 그림과 같이 거리가 5인 CP_C 를 최종적으로 찾는다.



ClosestPair의 수행 과정

- Line 6: $\text{dist}(CP_L)=10$, $\text{dist}(CP_C)=5$, $\text{dist}(CP_R)=15$ 중에서 가장 거리가 짧은 쌍인 CP_C 를 최근접 쌍의 점으로 리턴(최종해)



시간복잡도

□ 입력 S 에 n 개의 점이 있다면

- 전처리 (preprocessing) 과정으로서 S 의 점을 x -좌표로 정렬: $O(n \log n)$

□ Line 1

- S 에 3개의 점이 있는 경우에 3번의 거리 계산이 필요하고, S 의 점의 수가 2이면 1번의 거리 계산이 필요하므로 $O(1)$ 시간이 걸린다.

□ Line 2

- 정렬된 S 를 S_L 과 S_R 로 분할하는데, 이미 배열에 정렬되어 있으므로, 배열의 중간 인덱스로 분할하면 된다. 이는 $O(1)$ 시간 걸린다.

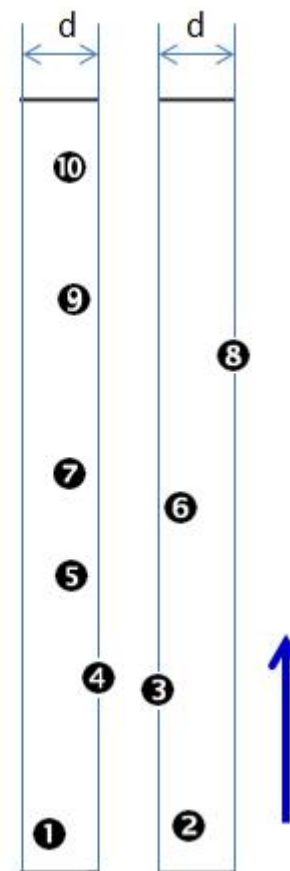
□ Line 3~4

- S_L 과 S_R 에 대하여 각각 ClosestPair를 호출하는데, 분할하며 호출되는 과정은 합병 정렬과 동일

시간복잡도

□ Line 5

- $d = \min\{\text{dist}(CP_L), \text{dist}(CP_R)\}$ 일 때 중간 영역에 속하는 점들 중에서 최근접 점의 쌍을 찾는다.
- 이를 위해 먼저 중간 영역에 있는 점들을 y-좌표 기준으로 정렬한 후에, 아래에서 위로 (또는 위에서 아래로) 각 점을 기준으로 거리가 d 이내인 주변의 점들 사이의 거리를 각각 계산하며, 이 영역에 속한 점들 중에서 최근접 점의 쌍을 찾는다.
- 따라서 y-좌표로 정렬하는데 $O(n \log n)$ 시간이 걸리고, 그 다음에는 아래에서 위로 올라가며 각 점에서 주변의 점들 사이의 거리를 계산하는데 $O(1)$ 시간이 걸린다. 왜냐하면 각 점과 거리 계산해야 하는 주변 점들의 수는 $O(1)$ 개이기 때문이다.



시간복잡도

□ Line 6

- 3개의 점의 쌍 중에 가장 짧은 거리를 가진 점의 쌍을 리턴하므로 $O(1)$ 시간이 걸린다.

□ ClosestPair 알고리즘의 분할과정은 합병 정렬의 분할과정과 동일

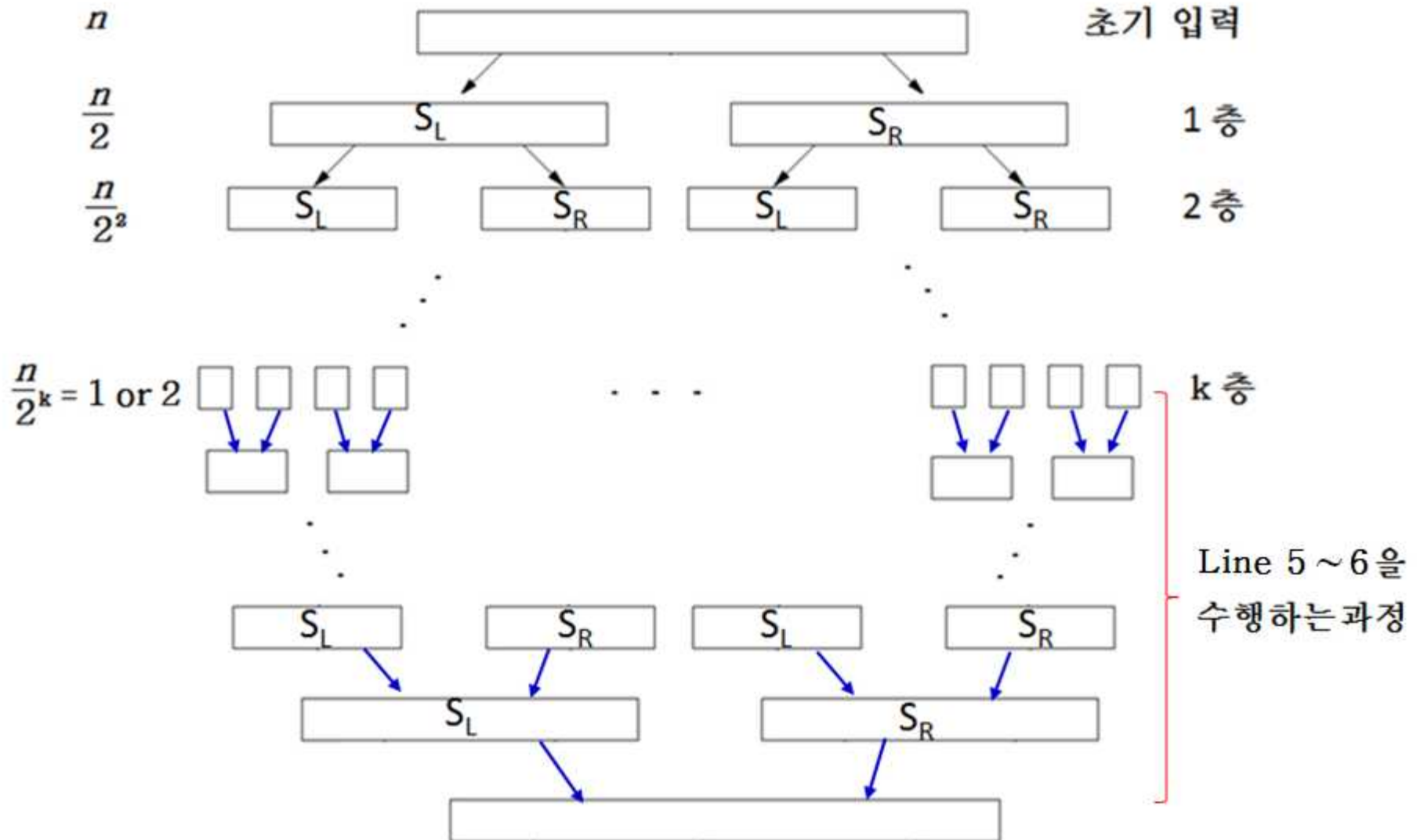
- 그러나 ClosestPair 알고리즘에서는 해를 취합하여 올라가는 과정인 line 5~6에서 $O(n\log n)$ 시간이 필요

□ 다음의 그림에서 k층까지 분할된 후, 층별로 line 5~6이 수행되는 (취합) 과정을 보여준다.

- 각 층의 수행 시간은 $O(n\log n)$ 이다.
- 여기에 층 수인 $\log n$ 을 곱하면 **$O(n\log^2 n)$** 이 된다.
(ClosestPair 알고리즘의 시간복잡도)

시간복잡도

■ 각 부분의
크기



응용

- ❑ 컴퓨터 그래픽스
- ❑ 컴퓨터 비전 (Vision)
- ❑ 지리 정보 시스템 (Geographic Information System, GIS)
- ❑ 분자 모델링 (Molecular Modeling)
- ❑ 항공 트래픽 조정 (Air Traffic Control)
- ❑ 마케팅 (주유소, 프랜차이즈 신규 가맹점 등의 위치 선정) 등

분할 정복 적용에 있어 주의할 점

□ 분할 정복이 부적절한 경우

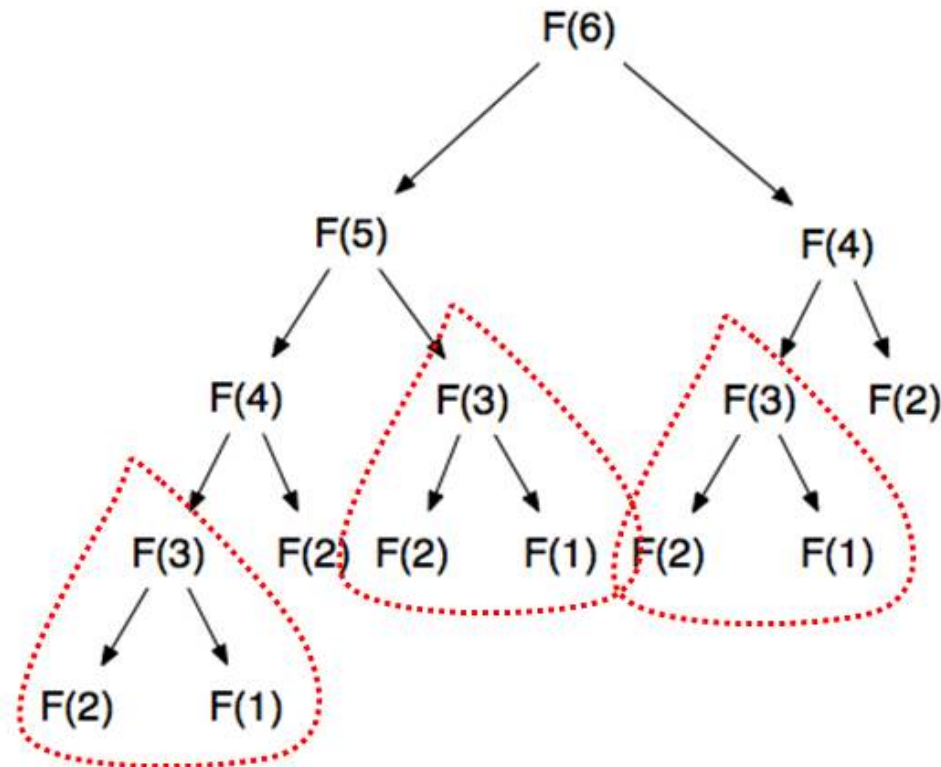
- 입력이 분할될 때마다 분할된 부분문제의 입력 크기의 합이 분할되기 전의 입력 크기보다 매우 커지는 경우이다.

□ n 번째의 피보나치 수를 구하기

- $F(n) = F(n-1) + F(n-2)$ 로 정의되므로 재귀 호출을 사용하는 것이 자연스러워 보이나, 이 경우의 입력은 1개이지만, 사실상 n 의 값 자체가 입력 크기인 것이다.
- 따라서 n 이라는 숫자로 인해 2개의 부분 문제인 $F(n-1)$ 과 $F(n-2)$ 가 만들어지고, 2개의 입력 크기의 합이 $(n-1) + (n-2) = (2n-3)$ 이 되어서, 분할 후 입력 크기가 거의 2배로 늘어난다.

피보나치 수

- 피보나치 수 $F(6)$ 을 구하기 위해 분할된 부분 문제들
 - $F(2)$ 를 5번이나 중복하여 계산해야 하고, $F(3)$ 은 3번 계산된다.



분할 정복 적용에 있어 주의할 점

- 주어진 문제를 분할 정복 알고리즘으로 해결하려고 할 때에 주의해야 하는 또 하나의 요소는 취합(정복) 과정이다.
- 입력을 분할만 한다고 해서 효율적인 알고리즘이 만들어지는 것은 아니다.
- 기하(geometry)에 관련된 다수의 문제들이 효율적인 분할 정복 알고리즘으로 해결된다.



Thank You